

Đại học Bách khoa Hà Nội

Trường Cơ Khí

Khoa Cơ Điện Tử



Báo cáo bài tập lớn

Môn: Vi xử lý

Đề tài: Thiết kế mạch điều khiển động cơ DC

Giảng viên hướng dẫn: **TS. Dương Văn Lạc**

Sinh viên thực hiện: **Nguyễn Quang Linh 20226662**

Mã Lớp: **157670**

Hà Nội, 06/2025

Mục lục

Chương 1: Phân tích hệ thống	3
1.1 Phân tích	3
1.2 Ý tưởng thiết kế	3
Chương 2: Thiết kế hệ thống.....	4
2.1. Kiến trúc hệ thống	4
2.2 Khối hiển thị	6
2.3 Khối xử lý trung tâm	6
2.4 Khối điều khiển	7
2.5 Khối I/O	7
Chương 3: Thiết kế chương trình điều khiển.....	8
3.1 Lưu đồ thuật toán.....	8
3.2 Mã nguồn	10
3.3 Mô phỏng	19
Chương 4: Thiết kế phần cứng	20
4.1 Mạch nguyên lý	20
4.2 Mạch 3D	21
4.3 Mạch thực tế.....	21
Chương 5: Tổng kết.....	22
5.1 Đánh giá hệ thống	22

Chương 1: Phân tích hệ thống

1.1 Phân tích

Để phát xung điều khiển động cơ ta cần sử dụng các bộ định thời (TIMER).

Để đọc tín hiệu từ chiết áp ta cần sử dụng bộ chuyển đổi tương tự - số (ADC).

Để xử lý nút bấm tốt nhất nên sử dụng ngắt ngoài (EXTERNAL INTERRUPT).

1.2 Ý tưởng thiết kế

Sử dụng ngắt ngoài TIMER0 để xử lý nút bấm Bật/Tắt động cơ.

Sử dụng ADC để đọc chiết áp.

Sử dụng TIMER1 INPUT CAPTURE để đếm xung Encoder.

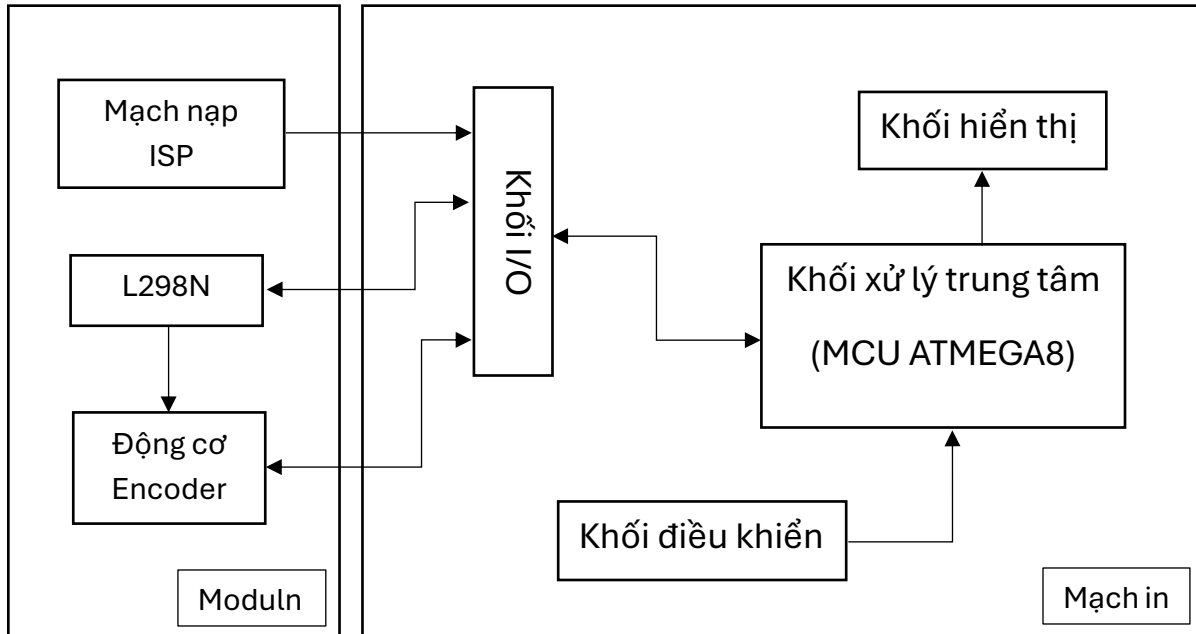
Sử dụng TIMER2 để phát xung PWM ở chế độ FAST PWM.

Sử dụng chế độ 4 bit để hiển thị thông số lên LCD. Kết nối song song LCD với MCU.

Các chân trên ATMEGA8	Mục đích sử dụng
PB5	Bật/tắt LED khi nút nhấn Start được bấm
PD2/INT0	Nút nhấn Start (Run)
PC0	Đọc duty qua ADC0
PB0	Input Capture qua TIMER1
PB2	Register Select (RS)
PD3	Enable (EN)
PD4-PD7	Chế độ 4 bit
PB1	IN1
PB3	Phát xung PWM cho động cơ
PB4	IN2

Chương 2: Thiết kế hệ thống

2.1. Kiến trúc hệ thống



- Mạch gồm các khối:
- + Khối xử lý trung tâm (MCU ATMEGA8).
- + Khối hiển thị.
- + Khối điều khiển
- + Khối I/O.

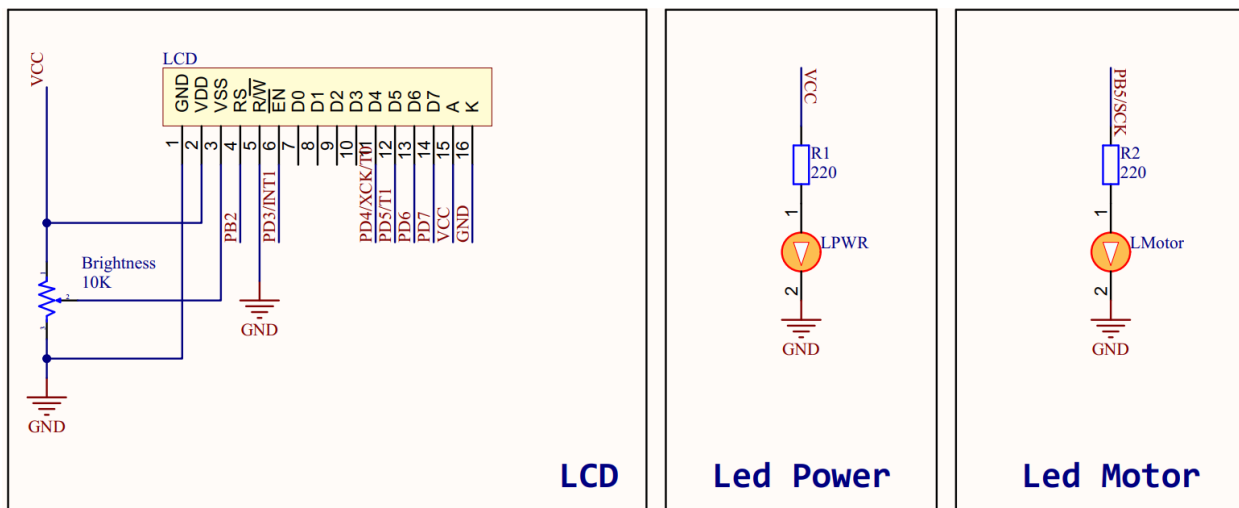
- Các linh kiện được tích hợp trong mạch:

Số thứ tự	Tên linh kiện	Số lượng
1	Atmega8	1
2	Chiết áp 2K	1
3	Chiết áp 10K	1
4	Header đực	
5	Điện trở 220	2
6	Điện trở 12K	2
7	Đèn LED	2
8	LCD 16x2	1
9	Nút nhấn	2
10	Thạch anh 16MHz	1
11	Tụ gốm 22pF	2

- Các moduln được sử dụng:

Số thứ tự	Tên moduln	Số lượng
1	Động cơ có gắn Encoder	1
2	L298N	1
3	Mạch nạp ISP	1

2.2 Khối hiển thị

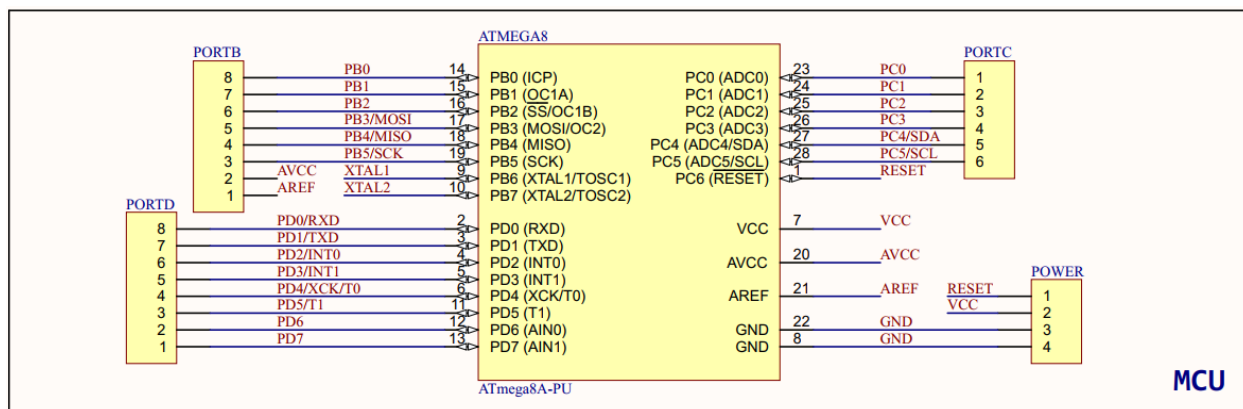


Khối hiển thị bao gồm: Màn hình LCD 16x2 và các Led debug như Led Power, Led Motor.

Chức năng của khối hiển thị là hiển thị các thông số RPM (Vòng/Phút) đo được từ Encoder, Duty Cycle (Tốc độ đặt cho động cơ) lên màn hình LCD 16x2.

Các Led Power, Led Motor có chức năng debug, báo hiệu cho người dùng khi có nguồn điện được cấp vào mạch hay động cơ.

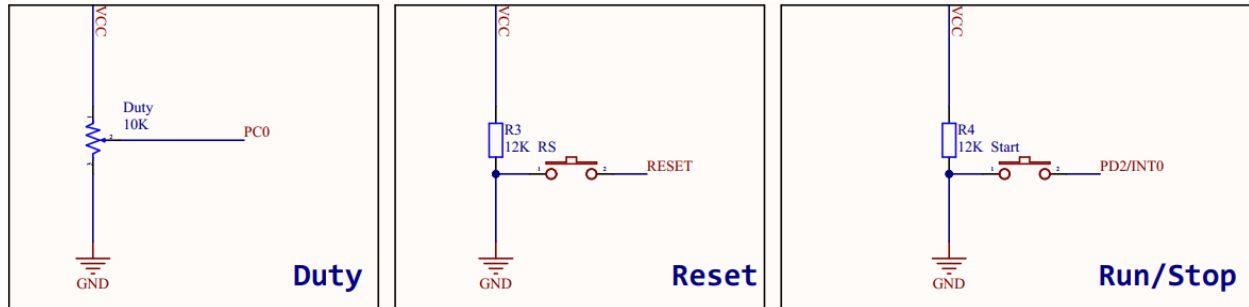
2.3 Khối xử lý trung tâm



Khối xử lý trung tâm bao gồm vi điều khiển ATMEGA8, bộ dao động thạch anh 16MHz và các header.

Chức năng chính của khối này là tiếp nhận, xử lý và gửi các tín hiệu.

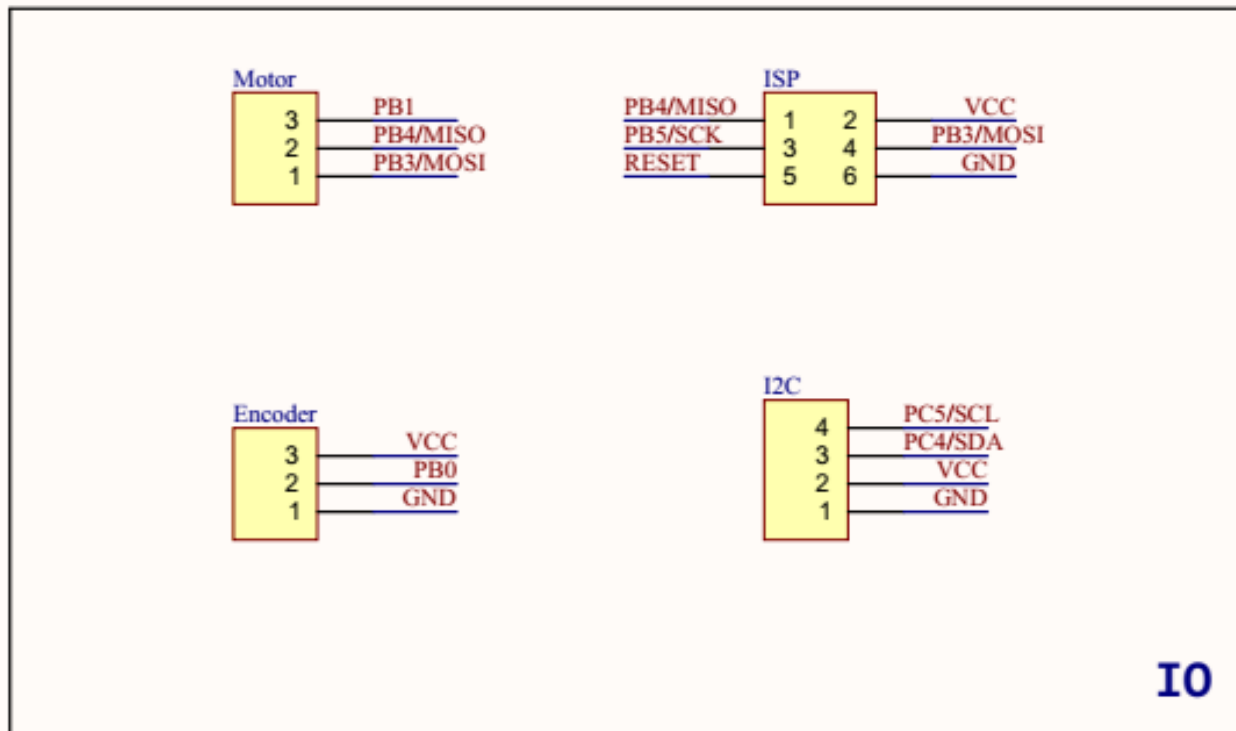
2.4 Khối điều khiển



Khối điều khiển bao gồm một chiết áp và các nút bấm.

Chức năng chính của khối này là điều chỉnh Duty Cycle, Bật/Tắt động cơ, Reset hệ thống.

2.5 Khối I/O

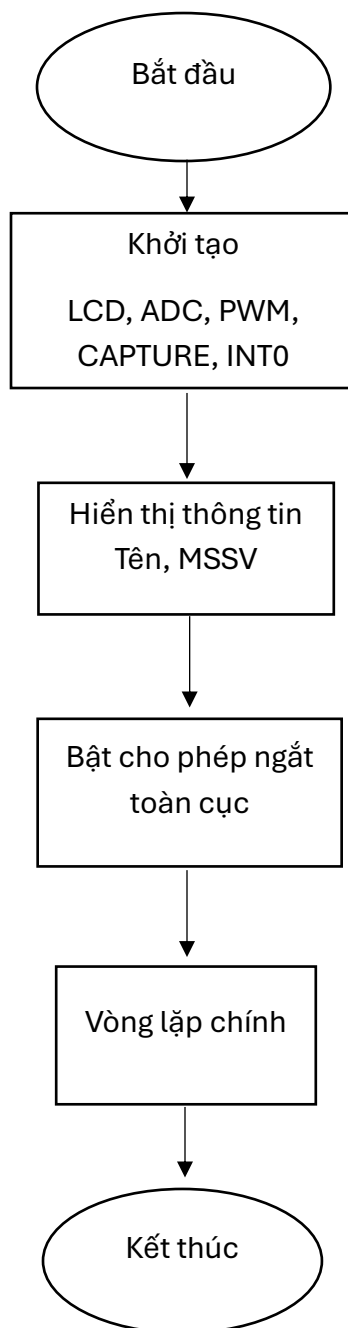


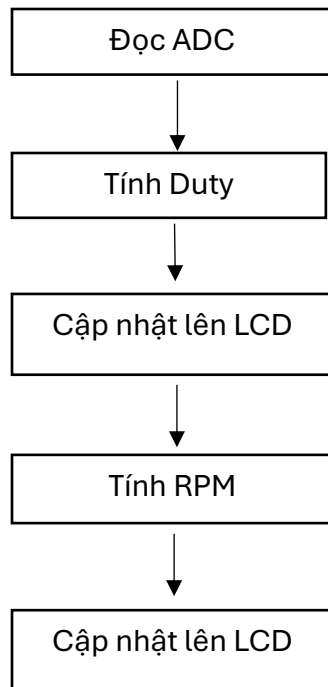
Khối I/O bao gồm các header có nhiệm vụ kết nối tín hiệu với các moduln ngoài.

Chương 3: Thiết kế chương trình điều khiển

3.1 Lưu đồ thuật toán

Lưu đồ thuật toán được chia thành 3 phần: Khởi tạo, Vòng lặp chính, Ngắt.





Ngắt khi nút Start được bấm

Ngắt INPUT CAPTURE

Ngắt tràn TIMER1

3.2 Mã nguồn

Khai báo xung thạch anh:

```
#define F_CPU 16000000UL
```

Thêm các thư viện cần thiết:

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdint.h>
#include <stdio.h>
#include <stdarg.h>
```

Định nghĩa dải giá trị của chiết áp:

```
// ADC calibration limits
#define ADC_MIN      15
#define ADC_MAX      1023
```

Định nghĩa Timeout khi không có xung:

```
// RPM measurement parameters
#define RPM_TIMEOUT  200    // ms until RPM display resets
```

Định nghĩa các cổng, thanh ghi kết nối với LCD:

```
// LCD pin definitions
#define LCD_DDR      DDRD
#define LCD_PORT      PORTD
#define LCD_EN       PD3
#define LCD_RS_DDR    DDRB
#define LCD_RS_PORT    PORTB
#define LCD_RS        PB2
```

Định nghĩa chân phát xung:

```
// PWM output
#define PWM_DDR      DDRB
#define PWM_PIN      PB3
```

Định nghĩa nút bấm:

```
// Button input (INT0)
#define BUTTON_DDR    DDRD
#define BUTTON_PORT    PORTD
#define BUTTON_PIN    PD2
```

Định nghĩa chân L298N:

```
// Motor driver pins
#define MOTOR_DDR    DDRB
#define MOTOR_IN1    PB1
#define MOTOR_IN2    PB4
```

Định nghĩa chân Led:

```
// Debug LED
#define LED_DDR      DDRB
#define LED_PIN      PB5
```

Khởi tạo các biến cờ và biến đếm:

```
// Global flags and counters
volatile uint8_t is_running = 0;
volatile uint32_t millis_counter; // Tracks time in milliseconds
volatile uint32_t last_capture_time; // Last time a capture event occurred

volatile uint32_t timer_overflow_count; // Accumulates timer overflows
volatile uint32_t first_time; // Time of first pulse in a sequence
volatile uint32_t period; // Time for 20 pulses (one revolution
if 20 pulses/rev)
volatile uint8_t pulse_count; // Number of pulses counted
volatile uint8_t ready; // Flag indicating RPM is ready to
display
```

Định nghĩa các hàm LCD:

```
// LCD low-level functions
static void LCD_Send4(uint8_t nibble) {
    LCD_PORT = (LCD_PORT & 0x0F) | (nibble & 0xF0);
    LCD_PORT |= (1 << LCD_EN);
    _delay_us(1);
    LCD_PORT &= ~(1 << LCD_EN);
}

static void LCD_Write(uint8_t value, uint8_t rs) {
    if (rs) {
        LCD_RS_PORT |= (1 << LCD_RS);
    } else {
        LCD_RS_PORT &= ~(1 << LCD_RS);
    }
    LCD_Send4(value & 0xF0);
    _delay_us(50);
    LCD_Send4((value << 4) & 0xF0);
    _delay_ms(2);
}
```

```

#define LCD_Command(cmd) LCD_Write(cmd, 0)
#define LCD_Data(dat)    LCD_Write(dat, 1)
#define LCD_Clear()      do { LCD_Command(0x01); _delay_ms(2); } while (0)

```

Khai báo các hàm:

```

// Utility function declarations
void LCD_Init(void);
void LCD_GotoXY(uint8_t row, uint8_t col);
void LCD_Print(const char *s);
void LCD_Printf(const char *fmt, ...);

void TIMER2_PWM_Init(void);
static inline void setDuty(uint8_t duty);

void TIMER1_CAPTURE_Init(void);

static inline void ADC_Init(void);
static inline uint16_t adcRead(uint8_t channel);

void Button_Init(void);
void Motor_Init(void);
void System_Init(void);

// Interrupt service routines
ISR(TIMER1_OVF_vect);
ISR(TIMER1_CAPT_vect);
ISR(INT0_vect);

```

Hàm main:

```

int main(void) {
    // Initialize peripherals
    LCD_Init();
    System_Init();

    // Configure I/O pins
    DDRC &= ~(1 << PC0);    // ADC input
    PORTC |= (1 << PC0);    // Enable pull-up
    LED_DDR |= (1 << LED_PIN); // LED as output
    PORTB &= ~(1 << LED_PIN); // LED off initially

    // Initialize system components with interrupts disabled
    cli();
    ADC_Init();
    Motor_Init();
    Button_Init();
    TIMER2_PWM_Init();
    TIMER1_CAPTURE_Init();
}

```

```

sei();

uint8_t last_duty = 0xFF; // Last set duty cycle
uint8_t prev_rpm = 0xFF; // Previous RPM value for display update
static uint16_t prev_adc = 0; // Previous ADC reading for edge case
handling

```

Vòng lặp chính:

```

while (1) {
    // Initialize RPM display on first run
    if (prev_rpm == 0xFF) {
        LCD_GotoXY(0, 0);
        LCD_Printf("RPM:%5d", 0);
        prev_rpm = 0; // Reset after initialization
    }
}

```

Chuyển tín hiệu từ ADC thành Duty Cycle.

```

// Read ADC and calculate duty cycle
uint16_t adc = adcRead(0);
uint8_t duty;

if (adc <= ADC_MIN) {
    if (prev_adc > 100) {
        duty = 100; // Potentiometer at maximum
    } else {
        duty = 0; // Potentiometer at minimum
    }
} else {
    // Map ADC range (15-1023) to duty cycle (0-100%)
    duty = (uint8_t)((adc - ADC_MIN) * 100UL / (ADC_MAX - ADC_MIN));
}
prev_adc = adc; // Update previous ADC value

```

Thực tế khi xoay hết một chu kỳ chiết áp ta nhận được hai dải giá trị là 0-1023 và dải không tuyến tính 0-15. Để ánh xạ cả hai dải này sang 0-100% Duty, ta sử dụng công thức:

$$\text{duty} = \frac{\text{adc} - \text{ADC_MIN}}{\text{ADC_MAX} - \text{ADC_MIN}} \times 100$$

Và cần xác định biên trái tương ứng với 0% Duty và biên phải tương ứng với 100% Duty.

Sau khi có được giá trị của Duty ta tiến hành cập nhật lên màn hình LCD nếu giá trị của nó thay đổi.

```
// Update PWM and LCD if duty changes
if (duty != last_duty) {
    last_duty = duty;
    LCD_GotoXY(1, 0);
    LCD_Printf("Duty:%4d%", duty);
    setDuty(duty);
}
```

Phần xử lý hiển thị giá trị RPM lên LCD:

```
// Update RPM display
if (!is_running || (millis_counter - last_capture_time > RPM_TIMEOUT))
{
    if (prev_rpm != 0) {
        prev_rpm = 0;
        LCD_GotoXY(0, 0);
        LCD_Printf("RPM:%5d", 0);
    }
    else if (ready) {
        ready = 0;
        // Calculate RPM: 60 * frequency, where period is time for 20
        pulses
        // Assuming 20 pulses per revolution, period/F_CPU is time per rev
        uint16_t rpm = (uint16_t)((60.0f * F_CPU) / (float)period);
        if (rpm != prev_rpm) {
            prev_rpm = rpm;
            LCD_GotoXY(0, 0);
            LCD_Printf("RPM:%5d", rpm);
        }
    }
}
return 0;
}
```

Nếu quá thời gian chờ xung, sẽ hiển thị giá trị 0. Trong trường hợp đã đếm đủ 20 xung, sẽ tiến hành tính toán RPM và cập nhật giá trị của nó lên LCD.

Định nghĩa các hàm:

```
// Function definitions

void LCD_Init(void) {
    LCD_DDR = 0xFF;           // LCD data port as output
    LCD_RS_DDR |= (1 << LCD_RS); // RS pin as output
    _delay_ms(20);
    LCD_Send4(0x30);
    _delay_ms(5);
    LCD_Send4(0x30);
    _delay_us(200);
    LCD_Send4(0x20);           // Set 4-bit mode
    _delay_us(200);
    LCD_Command(0x28);         // 4-bit, 2-line mode
    LCD_Command(0x0C);         // Display on, cursor off
    LCD_Clear();
}
```

```
void LCD_GotoXY(uint8_t row, uint8_t col) {
    uint8_t addr = (row ? 0x40 : 0x00) + col;
    LCD_Command(0x80 | addr);
}
```

```
void LCD_Print(const char *s) {
    while (*s) {
        LCD_Data(*s++);
    }
}
```

```
void LCD_Printf(const char *fmt, ...) {
    char buf[32];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buf, sizeof(buf), fmt, args);
    va_end(args);
    LCD_Print(buf);
}
```

Hàm phát xung trên TIMER2:

```
void TIMER2_PWM_Init(void) {
    PWM_DDR |= (1 << PWM_PIN); // PWM pin as output
    OCR2 = 128; // Initial 50% duty cycle
    // Fast PWM mode, clear OC2 on compare match, prescaler = 8
    TCCR2 |= (1 << COM21) | (1 << WGM21) | (1 << WGM20) | (1 << CS21);
}
```

Tần số phát xung:

$$f_{PWM} = \frac{f_{CPU}}{8 \times 256} = \frac{16\,000\,000}{8 \times 256} = 7812.5\,Hz = 7.8125\,kHz$$

Hàm điều chỉnh độ rộng xung:

```
static inline void setDuty(uint8_t duty) {
    if (duty > 100) {
        duty = 100; // Cap duty cycle at 100%
    }
    // Map duty 0-100% to OCR2 82-255 (~32%-100% PWM)
    // Possibly to ensure motor starts above a minimum threshold
    OCR2 = (uint8_t)(82 + (duty * 173UL) / 100);
}
```

Bằng việc thay đổi giá trị của thanh ghi OCR2 thì Duty Cycle sẽ được thay đổi. Thực tế động cơ có ngưỡng hoạt động của nó, dưới ngưỡng đó thì động cơ không chạy mà chỉ phát ra tiếng rít, để khắc phục điều này ta ánh xạ dải 0-100% Duty sang 82 (ngưỡng mà động cơ bắt đầu chạy) – 255.

Hàm đếm xung bằng INPUT CAPTURE trên TIMER1:

```
void TIMER1_CAPTURE_Init(void) {
    DDRB &= ~(1 << PB0);    // ICP1 (PB0) as input
    PORTB |= (1 << PB0);    // Enable pull-up
    TCNT1 = 0;
    timer_overflow_count = 0;
    // Timer1: no prescaler, input capture noise canceler
    TCCR1B = (1 << CS10) | (1 << ICNC1);
    TIMSK = (1 << TICIE1) | (1 << TOIE1); // Enable capture and overflow
    interrupts
}
```

Hàm thực hiện dịch vụ ngắt khi TIMER1 tràn và ngắt Input Capture:

```
ISR(TIMER1_OVF_vect) {
    timer_overflow_count += 0x10000UL; // Add 65536 per overflow
    millis_counter += 4; // Approx. 4ms per overflow at 16MHz
}
```

Ta tạo một bộ đếm thời gian để xác định được chính xác khoảng thời gian mà Input Capture không đếm được xung nào, từ đó reset giá trị RPM về 0.

```
ISR(TIMER1_CAPT_vect) {
    uint32_t now = timer_overflow_count + ICR1; // Total counts at capture
    last_capture_time = millis_counter;
    if (++pulse_count == 1) {
        first_time = now; // Start of pulse sequence
    } else if (pulse_count >= 20) {
        period = now - first_time; // Time for 20 pulses
        pulse_count = 0;
        first_time = now;
        ready = 1; // Signal RPM ready
    }
}
```

Trong hàm thực hiện dịch vụ ngắt Input Capture, ta ghi lại giá trị giữa xung đầu tiên và xung thứ 20 (đĩa Encoder có 20 xung), từ đó ta xác định được chu kỳ.

Định nghĩa hàm xử lý chiết áp:

```
static inline void ADC_Init(void) {
    ADMUX = (1 << REFS0); // AVCC as reference
    // Enable ADC, prescaler = 128 (125kHz at 16MHz)
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
}

static inline uint16_t adcRead(uint8_t channel) {
    ADMUX = (ADMUX & 0xF0) | (channel & 0x07); // Select channel
    ADCSRA |= (1 << ADSC); // Start conversion
    while (ADCSRA & (1 << ADSC)); // Wait for completion
    return ADC;
}
```

Định nghĩa hàm xử lý nút bấm:

```
void Button_Init(void) {
    BUTTON_DDR &= ~(1 << BUTTON_PIN); // Button pin as input
    BUTTON_PORT |= (1 << BUTTON_PIN); // Enable pull-up
    MCUCR |= (1 << ISC01); // Falling edge trigger
    MCUCR &= ~(1 << ISC00);
    GICR |= (1 << INT0); // Enable INT0
}
```

Hàm thực hiện dịch vụ ngắt khi nút bấm được bấm:

```
ISR(INT0_vect) {
    _delay_ms(10); // Simple debounce
    if (!(PIND & (1 << BUTTON_PIN))) { // Button still pressed
        PORTB ^= (1 << LED_PIN); // Toggle LED
        if (!is_running) {
            PORTB |= (1 << MOTOR_IN1);
            PORTB &= ~(1 << MOTOR_IN2); // Forward direction
        } else {
            PORTB &= ~((1 << MOTOR_IN1) | (1 << MOTOR_IN2)); // Brake mode
        }
        is_running = !is_running;
    }
}
```

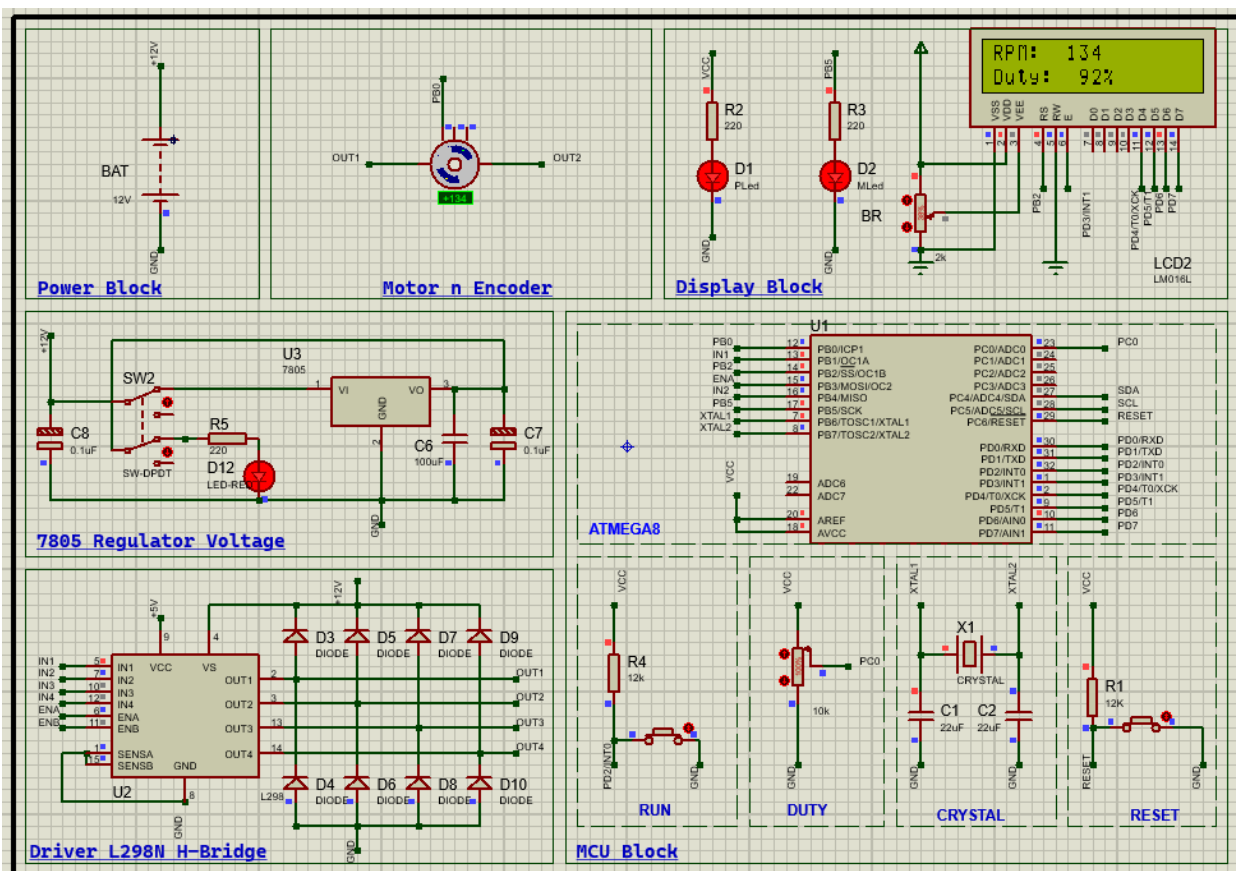
Hàm khởi tạo động cơ:

```
void Motor_Init(void) {
    MOTOR_DDR |= (1 << MOTOR_IN1) | (1 << MOTOR_IN2); // Motor pins as outputs
    PORTB &= ~(1 << MOTOR_IN1) | (1 << MOTOR_IN2); // Initially off
}
```

Hàm hiển thị tên, mã số sinh viên khi hệ thống bắt đầu chạy:

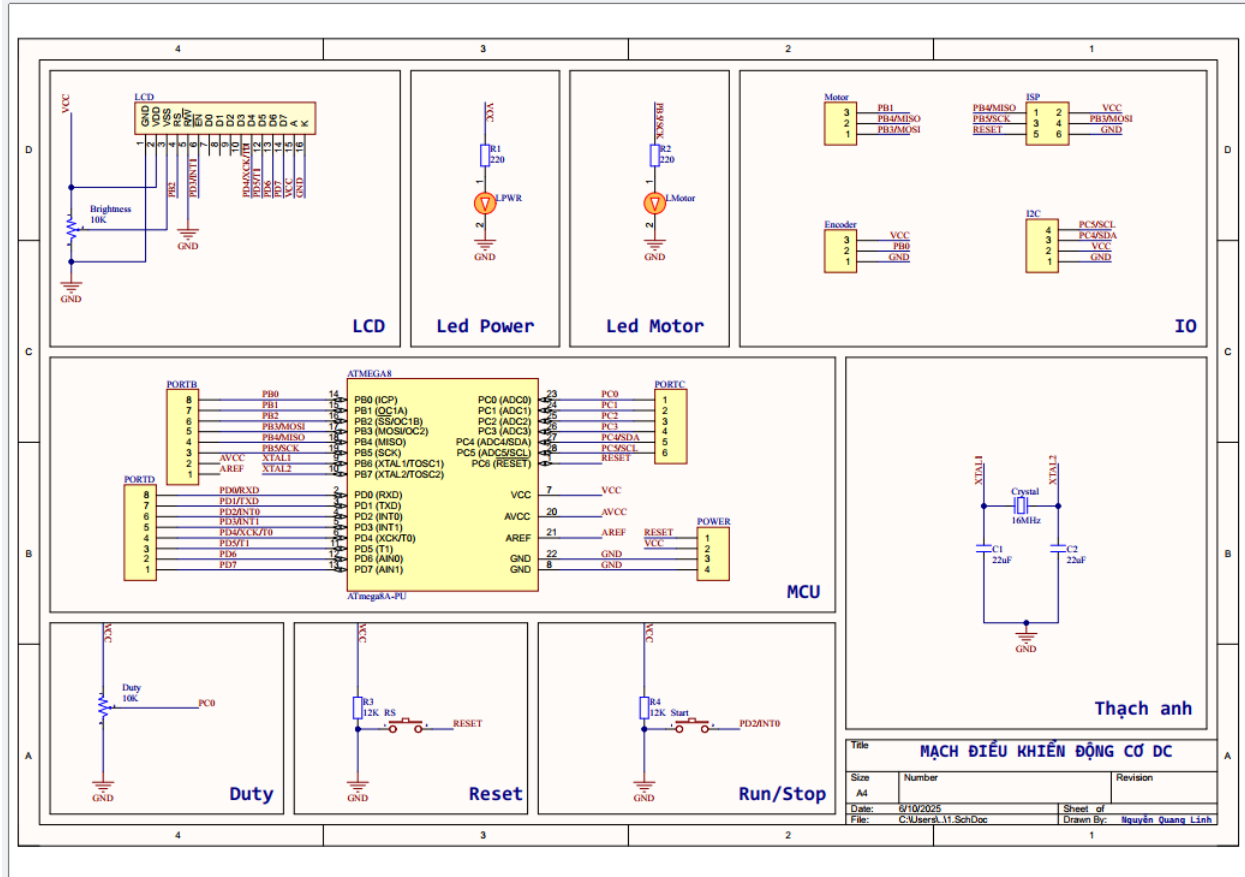
```
void System_Init(void) {
    LCD_GotoXY(0, 0);
    LCD_Print("Linh");
    LCD_GotoXY(1, 0);
    LCD_Print("20226662");
    _delay_ms(1000);
    LCD_Clear();
}
```

3.3 Mô phỏng

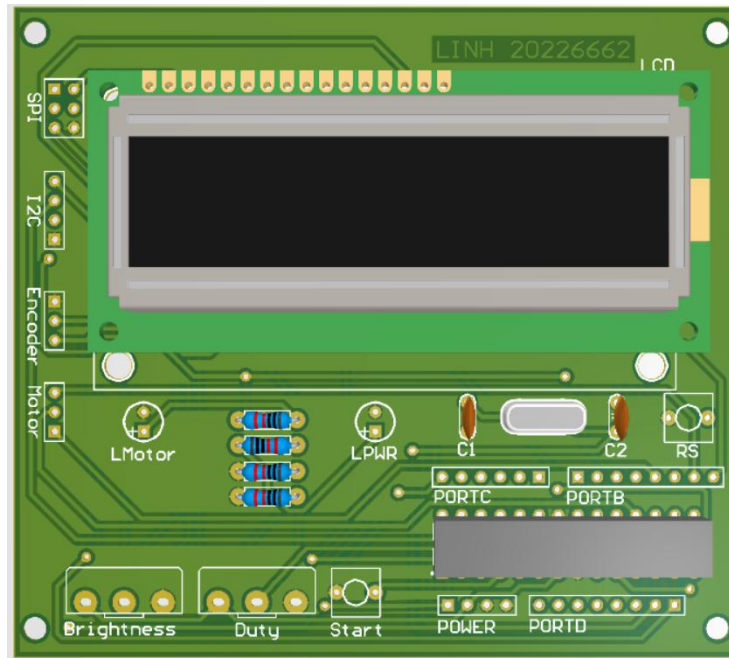


Chương 4: Thiết kế phần cứng

4.1 Mạch nguyên lý

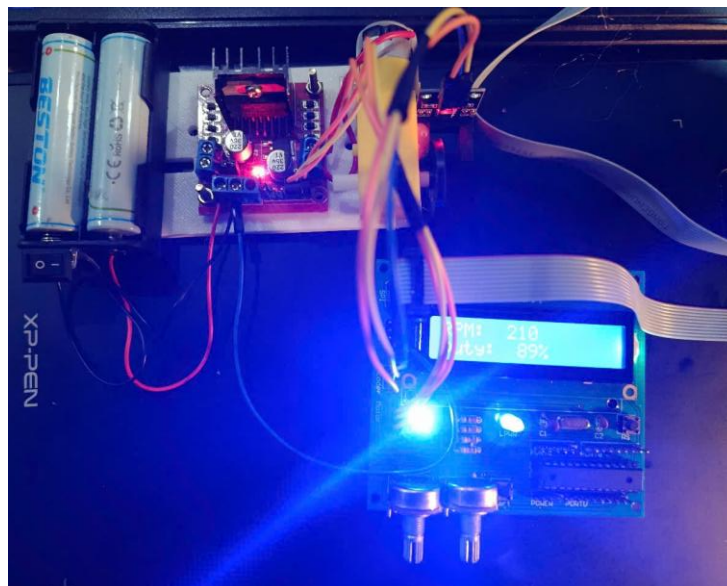


4.2 Mạch 3D



4.3 Mạch thực tế

Nhấn nút Start để bật động cơ, vặn núm xoay trên chiết áp để điều chỉnh Duty Cycle, nhấn nút RS để Reset hệ thống, vặn chiết áp BR để điều chỉnh độ tương phản của màn hình LCD.



Chương 5: Tổng kết

5.1 Đánh giá hệ thống

Ưu điểm: Mạch hoạt động ổn định, chính xác, dễ sử dụng, đáp ứng được các yêu cầu của đề bài.

Nhược điểm: Chức năng nạp code trực tiếp trên mạch bằng ISP chưa hoạt động được theo mong muốn. Mạch không có khối nguồn mà sử dụng moduln ngoài.

Cải tiến trong tương lai: Tích hợp khối nguồn vào trong mạch, sử dụng MOSFET để đóng ngắt động cơ, tích hợp các mạch bảo vệ, tích hợp thêm các cổng truyền thông USB,..